

Agent-Level Stochastic Orchestration for Deep Agent Network

Deep Agent Network Design Note

April 23, 2026

Abstract

This note reframes Deep Agent Network orchestration as an agent-level stochastic scheduling problem. The goal is not to create many named roles. The goal is to minimize time-to-quality by maintaining a dynamic task DAG, dispatching ready work across bounded agents, streaming artifacts upward, validating partial outputs, and pruning or duplicating branches under uncertainty. The core optimization target is quality and wall-clock time. Token cost, rework risk, and context loss remain extensible reward terms, but they should not dominate the first implementation. A central design requirement is robustness to misspecification: expected duration, expected quality gain, failure probability, uncertainty, and even the dependency graph itself are estimates, not facts.

1 Motivation

The earlier workflow-level view optimizes over fixed stages such as scouting, planning, execution, aggregation, validation, and delivery. That view correctly identifies a bottleneck: once aggregation or final delivery needs all prior evidence, those stages become serial barriers.

The stronger view moves optimization down to the agent/task level. DAN should maintain a dynamic queue of bounded tasks and route those tasks through agents as temporary compute slots. The useful structure is not a large catalog of personas. The useful structure is a scheduler over a task graph:

$$G_t = (V_t, E_t, A_t, C_t),$$

where V_t are tasks, E_t are dependency edges, A_t are artifacts, and C_t is the context/provenance ledger at time t .

2 Universal-Agent-Derived Scheduler

The scheduler should build on the universal-agent substrate rather than sit beside it as a separate black-box optimizer. The right split is:

- a universal scheduling agent proposes task decomposition, dependency revisions, context repair, branch continuation, cancellation, duplication, and finalization decisions;
- deterministic policy guards enforce capacity, hard dependencies, safety envelopes, confidence-bound pruning rules, and audit logging;
- worker reports, validators, reducers, and artifact ledgers feed back into the same recurrent universal-agent membrane.

This keeps DAN’s architecture coherent. The scheduler is not a hand-coded replacement for universal agents. It is a specialized universal-agent controller whose decisions are constrained by typed task contracts and observable queue mathematics.

The same applies one level above. The DAN-v2 conversation controller is also a universal-agent specialization: it frames the user turn, chooses the lane, sets the safety envelope, and defines the success contract. The scheduler is the next universal-agent specialization below it: it turns that framed objective into a dynamic task graph and queue policy. Controller, scheduler, worker, reducer, and validator should therefore differ by typed contract, not by being separate architectures.

3 Serial Barriers and Decomposition

Some stages are real barriers. Orchestration, final aggregation, validation, and delivery often have to observe earlier work before they can close the loop. The scheduler should not pretend those barriers disappear. It should reduce the amount of work waiting at the barrier by moving partial reduction, validation, and context repair earlier.

The main speedup target is therefore decomposition plus queueing inside the broad scouting, planning, execution, and validation work. For a decomposition into paths i and tasks j , the load-balancing problem is to choose a task split that reduces the slowest path:

$$\min_{\mathcal{D}} \max_i \sum_j T_{ij}.$$

This expression is incomplete without coordination overhead. A more operational target is:

$$\min_{\mathcal{D}, \pi} \left[\max_i \sum_j T_{ij} + C_{\text{coord}}(\mathcal{D}, \pi) + C_{\text{barrier}}(\mathcal{D}, \pi) \right],$$

where π is the queueing policy, C_{coord} captures handoff/reducer overhead, and C_{barrier} captures residual waiting at aggregation or final delivery. This prevents over-fragmentation: splitting work only helps when the reduced critical path is larger than the extra coordination cost.

4 Baseline Objective

For deterministic DAG scheduling with m parallel workers, a standard lower bound is:

$$T^* \geq \max \left(CP(G), \frac{W(G)}{m} \right),$$

where $CP(G)$ is the critical path length and $W(G)$ is total remaining work. In DAN, task runtimes and task values are uncertain, so the practical target should be:

$$\min_{\pi} \mathbb{E}[T_{\pi}(q_{\star})], \quad T_{\pi}(q_{\star}) = \inf\{t : Q_t \geq q_{\star}\},$$

where π is the scheduling policy, Q_t is current output quality, and $q_{\star}$ is the acceptable quality threshold. This is the cleanest product-level objective: minimize time-to-quality.

Equivalent constrained forms are also useful:

$$\max Q_T \quad \text{subject to} \quad T \leq B,$$

or

$$\min T \quad \text{subject to} \quad Q_T \geq q_\star.$$

These match the practical priority: quality first, then time, with token cost only as a later diagnostic or constraint.

5 Task Model

Each task should carry estimates, requirements, and output contracts:

$$\tau = (\text{deps}, \widehat{D}, \widehat{G}, \widehat{U}, \widehat{P}_f, \text{context}, \text{artifact}, \text{validator}).$$

Field	Meaning
$\widehat{D}$	expected duration distribution
$\widehat{G}$	expected quality gain
$\widehat{U}$	uncertainty or variance estimate
$\widehat{P}_f$	failure probability estimate
context	required context references and compression assumptions
artifact	typed output emitted by the task
validator	local test, rubric, critic, or deterministic checker

The scheduler’s actions include dispatching a task, splitting a task, duplicating a risky task, validating an artifact, cancelling a branch, improving context, asking for missing information, or finalizing from promoted artifacts.

6 Reward Shape

The first reward should prioritize quality and time:

$$R(a \mid s) = \widehat{g}_a - \lambda \widehat{d}_a,$$

where $\widehat{g}_a$ is the expected quality gain of action a and $\widehat{d}_a$ is expected latency. The framework should remain extensible:

$$R = w_q Q - w_t T - w_c C_{\text{token}} - w_r R_{\text{rework}} - w_x X_{\text{context-loss}}.$$

Initially, w_c , w_r , and w_x can be zero or diagnostic-only. The architecture should still record them so later policy experiments do not require a new event contract.

7 Context as a Resource

In LLM orchestration, context is not only transport. Context is a resource and a failure mode. A task may fail because the model is weak, but it may also fail because a compressed brief omitted one critical constraint.

DAN should therefore track:

- context completeness: whether all required facts are present

- context freshness: whether the context reflects the latest artifacts
- compression loss: what was removed or summarized away
- provenance: which task, source, or artifact produced each claim
- assumption debt: claims used before validation

This makes “send better context” a scheduler action. Sometimes that action is higher value than spawning another worker.

8 Robustness to Misspecification

The estimates above are misspecified by default. Duration distributions, quality gains, uncertainties, failure probabilities, and dependency edges are all learned approximations. A stable scheduler should avoid both over-pruning and noisy over-exploration.

8.1 Ambiguity Sets

Let θ represent the true task parameters and $\hat{\theta}$ the current estimate. Instead of optimizing only the point estimate, use an ambiguity set:

$$\theta \in \mathcal{U}(\hat{\theta}).$$

A robust scheduling rule can prefer:

$$a_t = \arg \max_a \min_{\theta \in \mathcal{U}(\hat{\theta})} R_\theta(a \mid s_t).$$

For a lightweight implementation, use confidence penalties:

$$S(a) = (\hat{g}_a - z_\beta \hat{\sigma}_a^g) - \lambda (\hat{d}_a + z_\alpha \hat{\sigma}_a^d).$$

This favors high expected gain, but penalizes uncertain gain and uncertain duration.

8.2 Conservative Pruning

Branch pruning should include slack. A branch b should be cancelled only when its optimistic value is clearly below the incumbent:

$$\text{UCB}_\beta(V_b) + \epsilon < \text{LCB}_\beta(V_{\text{best}}).$$

The slack term ϵ is important. It prevents early overkill when estimates are weak. DAN can also reserve a small exploration budget so at least one uncertain but plausible branch survives long enough to produce real evidence.

8.3 Dependency Uncertainty

The dependency graph can also be wrong. Edges should support states such as hard, soft, suspected, and disproven. A suspected dependency can block speculative finalization without preventing safe scouting or context acquisition. If later evidence contradicts the graph, the scheduler should be able to reopen a branch, add an edge, or demote an artifact.

8.4 Stability Controls

Robust orchestration needs hysteresis. The scheduler should not constantly reshuffle priorities because of tiny score changes. Useful controls include:

- minimum score margins before cancelling or redirecting work
- bounded redundancy for high-impact tasks
- periodic calibration from observed runtimes and validator outcomes
- straggler detection with split-or-replace decisions
- artifact promotion only after validator gates, not after confident prose

9 Streaming Reduction and Validation

The weak pattern is a global barrier:

scout all → wait → plan all → wait → execute all → wait → aggregate.

The stronger pattern is streaming:

worker output → local reducer → validator → promoted artifact → global synthesis.

Aggregation should be hierarchical and incremental. Validation should run before the end, not only as a final ceremony. Final delivery should assemble promoted artifacts and clearly mark unresolved assumptions.

10 Scheduler Loop

A first implementation can be expressed as:

1. Maintain the current task DAG, artifact set, and context ledger.
2. Estimate duration, quality gain, uncertainty, and failure risk for ready tasks.
3. Dispatch the highest robust-priority tasks subject to dependency and capacity limits.
4. Stream back artifacts, assumptions, blockers, and confidence.
5. Validate artifacts and promote only those that pass the relevant gate.
6. Recompute priorities and update the graph.
7. Cancel, split, duplicate, or continue branches using conservative pruning.
8. Return once the quality threshold is reached or the remaining upper bound cannot justify more work.

11 Design Principle

DAN should feel less like roleplay and more like a distributed build system for cognition. Agents are temporary compute cells. The task graph, queueing policy, context ledger, artifact validators, and reducer hierarchy are the organism.